

Sri Lanka Institute of Information Technology

Assignment 01

SE3092 – Platform Based Developments

Personal Finance Management System for Android — SRS

Student ID	Name
IT23657496	Kekulanthale K. M. N. Y.
IT23632578	Isuranga A.G.A
IT23655652	Meddegama B. T. M. R. N. B.

Introduction

1.1 Purpose

This document specifies the functional, non-functional and data requirements, technical constraints, use cases and acceptance criteria for an Android personal finance app designed for Kavindu Silva. It replaces the four tracking tools that failed him with one structured, low-friction, multi-source and multi-currency record.

The SRS is deliberately scenario-specific. Its central objective is to make Kavindu's **actual** income, **actual** spending and progress toward the MacBook goal visible, without the daily data-entry burden that caused every earlier tool to be abandoned.

1.2 Scope

The app shall let an authenticated user record and review:

- income from salary, freelance milestones, Google AdSense (USD) and cryptocurrency, each with its own rules
- expenses across main-bank card, cash and secondary-account channels, with committed/discretionary classification
- transfers between his own accounts, including transfers to a dedicated savings account
- Binance trade-history CSV imports, from which realised crypto profit/loss is calculated a monthly close that reconciles recorded figures against real account balances so the books balance
- savings-goal progress linked to daily spending, with reminders and re-engagement after missed days

The app shall **not** connect to bank accounts, move money, call the Binance API or import bank transactions. All data is user-entered, generated from user-configured recurring templates, or imported from a CSV file the user selects.

1.3 Intended audience

The SE3092 development team, module evaluators and the demonstration panel.

1.4 Definitions and abbreviations

Term	Definition
Account	A place money sits: Main bank, Secondary bank, Cash wallet, Savings, Binance
Transfer	Money moved between two of the user's own accounts; never income or expense
Confirmed income	Income recorded as actually received in an account
Expected income	Income anticipated but not yet confirmed (salary on the 25th, an invoiced milestone)

Term	Definition
Committed expense	A cost the user is bound to pay: rent, utilities, gym, subscriptions
Discretionary expense	A cost chosen at the time: dining, coffee, rides, food delivery, shopping
Realised crypto P&L	Profit or loss locked in when a crypto position is sold, calculated from imported trades
Portfolio snapshot	A manually recorded total portfolio value at a point in time; not P&L
Monthly close	End-of-month step comparing calculated balances with real balances
Unaccounted amount	Difference found at monthly close, recorded so debits and credits balance
Backfill / catch-up	Entering or reconciling missed days without dayby-day entry
Effective rate	LKR actually credited ÷ USD amount for a foreigncurrency receipt
FIFO	First-in, first-out cost basis for crypto sales
MVVM, DI, FR, NFR	Model–View–ViewModel; Dependency Injection; Functional / Non-Functional Requirement

1.5 Document conventions

"Shall" is mandatory; "should" is recommended; "may" is optional. Each requirement has one unique ID. Priority uses **Must / Should / Could** (MoSCoW). Each requirement states how it will be verified.

User and Problem Context

Kavindu earned over LKR 1.6 million in 12 months but holds LKR 11,200 in savings (about 0.7%). He cannot explain where the rest went.

2.1 Persona: Kavindu Silva

Attribute	Scenario detail
Age / role	25, Junior Software Engineer at a Colombo startup; graduated 12 months ago
Salary	LKR 120,000–140,000, paid on the 25th; base plus performance bonus
Freelance	React and WordPress builds for local SMEs; LKR 20,000–90,000 per project, milestone-based
AdSense	Paid monthly in USD; LKR 3,500–19,000 equivalent
Crypto	USDT, ETH and altcoins on Binance; big gain in Q1, material loss in Q3
Goal	MacBook Pro M4, about LKR 490,000, within 12 months
Behaviour	Driven and technical; impulsive with discretionary spending; optimistic about income, avoidant about expenses
Data-entry tolerance	Low; every tracking attempt failed within two weeks

2.2 Income situation

- **Salary:** arrives on the 25th but varies; he rarely checks the payslip and cannot recall the exact deposit.
- **Freelance:** paid irregularly by milestone; he has twice forgotten to chase invoices and once could not match a payment to a project.
- **AdSense:** paid in USD and converted at the bank's rate on the transfer day; he never records what actually arrives.
- **Crypto:** active Binance trading with no running position; he cannot state his year-todate result.
- **Overall:** he estimates monthly income at "LKR 160,000–210,000" and has never validated it.

2.3 Expense situation

- Spending runs through bank card, cash withdrawals, PickMe, UberEats, a gym autodebited from a **secondary account**, and two subscriptions he meant to cancel six months ago.
- Rent is LKR 34,000; utilities move the real monthly figure.
- One month of coffee and casual dining reached LKR 24,000 without him noticing. He has no categories and no idea of his committed-to-discretionary ratio.

2.4 Savings goal

From LKR 11,200 saved, he needs LKR 478,800 more. Over 12 months that is about **LKR 39,900 per month**, roughly 20–25% of his estimated income. The app must calculate this from stored values, not hard-code it.

2.5 Previous failed tracking attempts

Tool	What happened	Root cause	System response
Google Sheet "MacBook Fund"	Formulas broke on row insertion; abandoned after 13 days plus a 3-week gap	High manual effort; fragile formulas; gaps felt like failure	Calculations in code; 3tap entry; catch-up mode and reminders
Play Store budget app	Uninstalled in 9 days: USD only, no sync, one income, fixed budget	Data model too narrow	Four-source, multicurrency income; Firestore sync; no fixed budget required
Tool	**What happened**	**Root cause**	**System response**
Notes app	Pasted bank exports became unreadable in 4 days	No structure	Structured records with category, account and date
Bank spending summary	Card only; missed cash and secondary account; PickMe tagged "online retail"	Partial coverage, no user control	All accounts in one place; editable categories

2.6 Core problems identified

#	Problem	Evidence in scenario
P1	Financial invisibility	Cannot explain where LKR 1.6M went
P2	Income is estimated, never confirmed	Salary amount unknown; AdSense never recorded
P3	Freelance reconciliation failure	Forgotten invoices; unmatched payment
P4	Crypto result unknown	No running position or YTD answer
P5	Expense fragmentation	Card, cash, apps and secondary account never seen together
P6	No committed/discretionary view	LKR 24,000 dining surprise
P7	Tracking friction	Every tool abandoned within two weeks
P8	No recovery after gaps	Sheet abandoned after a 3-week break
P9	Goal disconnected from behaviour	Does not know the monthly saving he needs or achieves
P10	No way to verify the books	Has never matched recorded figures against real balances

Requirements Analysis

The brief poses five analytical questions. Each answer below becomes a group of requirements in Sections 5 and 6.

3.1 What must the system understand about income structure?

An amount alone is meaningless for Kavindu. Each income record needs its source, currency, timing, certainty and link to a project or schedule.

Dimension	Scenario implication	System response
Source	Four channels behave differently	Separate source types with sourcespecific fields and flows
Certainty	Salary and invoices are expected before they are received	Expected vs Confirmed status; only Confirmed counts in totals
Currency	AdSense is USD; bank rate differs from market rate	Store USD amount, LKR actually credited and the effective rate
Timing	Salary on the 25th; milestones and payouts irregular	Expected dates, due dates and payday prompts
Dependability	Crypto can lose money	Classify income as **dependable** (salary base), **variable** (bonus, freelance, AdSense) or **speculative** (crypto); plan savings on dependable income only
Verification	His own estimate was never checked	Monthly close compares the recorded total with real balances

3.2 What do the four failures share, and how is that prevented?

All four tools were either too manual, too narrow, too unstructured or too partial. A fifth shared failure: **a gap in data made the tool feel broken, so he stopped**. The design responses are:

- **System-owned calculations** in repositories/use cases, never user-edited formulas.
- **A data model that fits him**: accounts, transfers, four income sources, multi-currency.
- **Structured records** with mandatory category, account and date.
- **Full coverage**: every account he uses, including cash and the secondary account.
- **Forgiving recovery**: reminders at 3, 7 and 14 days, then a catch-up mode that reconciles to current balances instead of demanding every missed transaction.
- **Architecture for reliability**: offline-first Room cache, Firestore sync, so the app always opens fast.

3.3 How should goal progress feel real?

- Show **saved / target / percentage**, **required per month** and a **projected completion date** based on his last 3 months' actual savings.
- Prompt a **savings transfer on salary confirmation** ("pay yourself first"), recorded as a transfer to the Savings account.

- Translate discretionary spending into goal impact, e.g. "This month's dining out = 6 days of delay."
- Notify milestones at 25%, 50%, 75% and 100%.

3.4 How should expense entry minimise friction, and what must a record hold?

Entry is amount-first: keypad → category chip → save, with account and date defaulted. Categories carry a default committed/discretionary flag, so classification costs no extra tap. Recurring items (rent, gym, subscriptions) are generated automatically for one-tap confirmation.

Field	Why it is needed at month end
Amount + currency	Totals and conversion
Date	Daily, weekly and monthly analysis
Category	Where money goes
Account	Cash and secondary-account visibility; balance reconciliation
Committed/discretionary	The ratio he has never seen
Recurring link	Subscription audit
Note (optional)	Context without slowing entry

3.5 Which views give self-awareness within the first week?

1. **Dashboard:** confirmed income, spending, net, safe-to-spend and goal ring.
2. **Spending by category** for the period.
3. **Committed vs discretionary** split.
4. **Goal progress** with required vs actual monthly saving.
5. **Weekly summary** after day 7, even with partial data.

3.6 Derived design principles

1. **Accuracy over estimation**: confirmed amounts replace guesses.
2. **Friction is the enemy**: a routine expense takes 3 taps or fewer.
3. **Gaps are recoverable**: catch-up and reconciliation, never guilt.
4. **Everything links to the goal**.
5. **One place**: all income, expenses, transfers and balances in one ledger.
6. **Manual capture only**: no bank integration; the user stays in control.
7. **Local-first**: entry and viewing work offline.

Overall System Description

4.1 Product perspective

A standalone Android app. Firebase Authentication manages identity, Firestore is the primary cloud store with real-time listeners on the dashboard, and Room caches data locally so the app works offline. External data enters only by manual entry, userconfigured recurring templates, or a Binance CSV the user selects.

4.2 User characteristics

Kavindu is technically capable but will not maintain anything that feels like work. The app must not require spreadsheet skills, accounting knowledge or a fixed monthly budget.

4.3 Product functions

Module	Purpose
Authentication (AUTH)	Register, sign in, isolate each user's data
Accounts and transfers (ACC)	Model bank, cash, secondary, savings and Binance accounts; record transfers
Income (INC)	Salary, AdSense/foreign currency, expected vs confirmed
Freelance (PROJ)	Clients, projects, milestones, invoices, overdue follow-up
Crypto (CRY)	Binance CSV import, realised P&L, manual snapshots
Expenses (EXP)	3-tap entry, categories, classification, backfill, correction
Recurring (REC)	Rent, gym, subscriptions; subscription audit
Savings goal (GOAL)	Target, required saving, projection, savings transfer prompt
Monthly close (CLOSE)	Reconcile against real balances; balanced monthly statement
Dashboard (DASH)	One consolidated, real-time view
Engagement (ENG)	Reminders, re-engagement, catch-up mode
Offline and sync (OFF)	Local-first entry with background sync
Validation (VAL)	Field validation, duplicates, empty and error states

4.4 Assumptions

- The user can export trade history from Binance as a CSV file.
- The user can see real account balances in his banking app at month end.
- The savings account is a real bank account; the app records transfers but cannot perform them.
- A public exchange-rate source may suggest USD/LKR rates; manual entry is always allowed.
- The user is periodically online for sync.
- The user grants notification permission (required on Android 13+).

4.5 Dependencies

Firebase Authentication; Cloud Firestore; Room; Android WorkManager and notifications; Android Storage Access Framework (CSV picker); optional exchange-rate REST API.

4.6 Constraints

ID	Constraint
C-01	100% Kotlin in the application layer; Gradle Kotlin DSL
C-02	Jetpack Compose with Material Design 3; no XML layouts
C-03	Navigation Compose, single-activity architecture
C-04	Strict MVVM: UI → ViewModel → Repository → data source
C-05	StateFlow/LiveData exposed read-only; Coroutines with viewModelScope
C-06	Hilt (preferred) for dependency injection
C-07	Firebase Auth email/password mandatory
C-08	Firestore primary store; Security Rules submitted
C-09	Room for offline caching
C-10	Min SDK 26, target SDK 34+
C-11	No automatic bank data import
C-12	APK connects to a live Firebase project

4.7 Out of scope

- Bank, card, AdSense or Binance API connections; moving money between accounts
- Live crypto prices, unrealised gains, order execution
- Tax calculation
- Shared or multi-user budgets
- Financial or investment advice
- iOS and web clients

Functional Requirements

Each requirement has a unique ID, a business rule, a MoSCoW priority and a verification method. "Source" links to the problems in Section 2.6.

5.1 Authentication and user management (AUTH)

ID	Requirement	Rule	Priority	Verification	Source
FRAUTH-001	The system shall let a new user register with email and password via Firebase Authentication.	Inputs validated before submission; errors shown without technical detail	Must	Register with valid and invalid credentials	—
FRAUTH-002	The system shall sign in an existing user and keep the session across restarts until sign-out.	Unauthenticated users cannot reach financial screens	Must	Sign in, restart app, confirm session	—
FRAUTH-003	The system shall support password reset by email.	—	Must	Trigger reset email	—
FRAUTH-004	The system shall store every financial record under the signed-in user's UID and block cross-user access.	Enforced by Firestore Security Rules, not UI filtering	Must	Rules emulator test with users A and B	—
FRAUTH-005	The system shall let the user sign out and remove access to cached financial data.	—	Must	Sign out; protected screens inaccessible	—
FRAUTH-006	The system shall run a short onboarding capturing accounts, salary day, rent, known subscriptions and the savings goal.	Every step skippable	Should	Complete onboarding in under 2 minutes	P7
FRAUTH-007	The system may support Google Sign-In.	Bonus credit only	Could	Sign in with Google	—

5.2 Accounts and transfers (ACC)

ID	Requirement	Rule	Priority	Verification	Source
FRACC-001	The system shall let the user create accounts of type Bank, Cash, Savings or Crypto exchange, each with name, currency and opening balance.	Defaults: Main bank, Secondary bank, Cash, Savings, Binance	Must	Create each type	P5
FRACC-002	The system shall record transfers between the user's accounts.	A transfer is never counted as income or expense	Must	ATM withdrawal (Main → Cash) leaves monthly spend unchanged	P5
FRACC-003	Money sent to the secondary account shall be a transfer; gym and subscriptions paid from it shall be expenses.	Prevents double counting	Must	Record both; total spend counts gym once	P5
FRACC-004	The system shall show a running balance per account from opening balance and all entries.	—	Must	Balance equals opening + credits − debits	P1

5.3 Income (INC)

ID	Requirement	Rule	Priority	Verification	Source
FR-INC-001	The system shall record income from Salary, Freelance, AdSense and Crypto as distinct source types.	Source is a field, never inferred from text	Must	One entry per source; filter by source	P1
FR-INC-002	Each income record shall store amount, currency, date, account, status and LKR reporting amount.	Amount > 0	Must	Inspect stored record	P2
FR-INC-003	Income shall have status Expected or Confirmed.	Only Confirmed counts in income totals	Must	Mixed entries; dashboard totals differ correctly	P2
FR-INC-004	The system shall store a salary profile: pay day (default 25th), base amount, expected bonus range and receiving account.	Pay day editable	Must	Configure profile	P2
FR-INC-005	On pay day the system shall create an Expected salary entry and notify the user to confirm the exact amount credited.	Prompt only; no bank access	Must	Simulate the 25th; confirm entry	P2
FR-INC-006	Salary shall be recorded as base plus bonus, with monthly bonus history.	Values outside 120k–140k allowed	Should	Log 3 months; view history	P2
FR-INC-007	The system shall record foreign-currency income with original amount, currency and receipt date.	—	Must	Enter USD 45 AdSense payout	P2
FR-INC-008	The system shall suggest the USD/LKR rate for the receipt date and let the user enter the LKR actually credited; it shall store the effective rate.	Rate > 0; manual fallback when offline	Must	Enter credited LKR; effective rate stored	P2
FR-INC-009	Stored LKR values shall never be recalculated when rates change later.	—	Must	Change rate source; old entry unchanged	P2
FR-INC-010	The system shall remind the user on the expected AdSense payout date.	User-configurable	Should	Reminder fires	P2
FR-INC-011	The system shall tag each source as dependable, variable or speculative and show monthly income in those three bands.	Salary base = dependable; bonus, freelance, AdSense = variable; crypto = speculative	Should	Income view shows bands	P2

5.4 Freelance projects and milestones (PROJ)

ID	Requirement	Rule	Priority	Verification	Source
FRPROJ-001	The system shall let the user create clients and projects with name, work type, total value and dates.	Value > 0 when given	Must	Create project	P3
FRPROJ-002	A project shall hold one or more milestones with amount, invoice date, due date and status.	Status: Planned, Invoiced, Paid, Overdue, Cancelled	Must	Add 3 milestones	P3

ID	Requirement	Rule	Priority	Verification	Source
FRPROJ-003	Recording a freelance payment shall require selecting the milestone it settles.	Cannot link to a nonexistent milestone	Must	Payment linked; milestone set to Paid	P3
FRPROJ-004	The system shall mark an Invoiced milestone Overdue after its due date and send a follow-up reminder.	Reminder repeats weekly until Paid	Must	Pass due date; reminder fires	P3
FRPROJ-005	The system shall show total outstanding (invoiced, unpaid) on the dashboard and per project.	Outstanding never counts as Confirmed income	Must	Outstanding equals unpaid milestones	P3
FRPROJ-006	The system shall support partial payments against a milestone.	Remaining stays outstanding	Should	Pay 50%; balance outstanding	P3

5.5 Cryptocurrency (CRY)

ID	Requirement	Rule	Priority	Verification	Source
FRCRY-001	The system shall import a Binance trade-history CSV selected by the user.	Uses the Android file picker	Must	Import sample export	P4
FRCRY-002	The system shall validate columns and show a preview of rows read, rejected and duplicate before saving.	Missing columns listed by name	Must	Import malformed file; clear error	P4
ID	**Requirement**	**Rule**	**Priority**	**Verification**	**Source**
FRCRY-003	The system shall skip already-imported trades.	Match on trade ID, or date + pair + quantity	Must	Re-import same file; zero new rows	P4
FRCRY-004	The system shall calculate realised P&L per sale using FIFO cost basis, in USDT, converted to LKR at the trade date's rate.	Fees deducted	Must	Known buy/sell pair gives expected P&L	P4
FRCRY-005	The system shall show realised P&L by month and year-to-date, including losses.	—	Must	Q1 gain and Q3 loss both shown	P4
FRCRY-006	The system shall let the user record manual portfolio snapshots and show change between them.	Labelled "portfolio change", never "P&L"	Should	Two snapshots; change shown	P4
FRCRY-007	LKR moved to and from Binance shall be transfers; the system shall show net cash in vs out.	—	Should	Deposit and withdrawal; net shown	P4
FRCRY-008	Crypto P&L shall be excluded from dependable income used for savings planning.	—	Must	Large crypto gain does not change required saving	P4, P9

5.6 Expenses (EXP)

ID	Requirement	Rule	Priority	Verification	Source
FREXP-001	The system shall let the user add a routine expense in 3 taps or fewer: amount → category → save.	Account and date defaulted; editable before save	Must	Count taps from + button	P7

ID	Requirement	Rule	Priority	Verification	Source
FREXP-002	Each expense shall store amount, currency, date, category, account and committed/discretionary flag.	Amount > 0; past dates allowed	Must	Inspect stored record	P5, P6
ID	**Requirement**	**Rule**	**Priority**	**Verification**	**Source**
FREXP-003	The system shall provide default categories: Rent, Utilities, Groceries, Food out, Coffee, Food delivery, Rides, Gym, Subscriptions, Shopping, Other.	Each pre-flagged committed or discretionary; user can add and edit	Must	Category list present	P6
FREXP-004	The system shall show the 5 most-used categories first as one-tap chips.	Ordered by recent use	Should	Chips reorder with use	P7
FREXP-005	The user shall be able to correct an expense's category, account, classification or date.	Analysis recalculates	Must	Fix a mislabelled PickMe ride	P5
FREXP-006	The system shall support optional notes.	Never required	Could	Save with and without note	—
FREXP-007	The system shall let the user enter several past expenses in one backfill screen.	Each keeps its own date	Must	Backfill 5 entries over 3 days	P8
FREXP-008	The system may provide a homescreen widget or quick-settings tile for adding an expense.	—	Could	Add from widget	P7

Non-Functional Requirements

Every NFR is measurable so it can be shown in the live demo.

ID	Category	Requirement	Priority	Verification	Why (scenario)
NFR-USE001	Usability	A routine expense takes ≤ 3 taps and ≤ 5 seconds from the home screen.	Must	Timed task	Sheet entry overhead
NFR-USE002	Usability	A first-time user completes the first expense within 10 seconds without guidance.	Should	Usability walkthrough	Low tolerance for learning
NFR-USE003	Usability	Catch-up after a 14-day gap takes ≤ 2 minutes.	Must	Timed task	Sheet abandoned after gap
NFR-USE004	Usability	The dashboard answers "earned, spent, saved this month?" without scrolling on a 6-inch screen.	Must	Screenshot review	Financial invisibility
NFR-USE005	Usability	Missed days are never shown as failures; notification wording is nonjudgemental.	Must	Content review	Avoidant personality
NFR-USE006	Usability	Material Design 3; dark mode; system font scaling up to 200%.	Should	Device settings test	Brief §5
NFR-PERF-001	Performance	Dashboard renders cached data within 1 second of opening, before network data arrives.	Must	Benchmark on API 26 device	App must be pleasant to open daily
NFR-PERF-002	Performance	A saved entry appears on the dashboard within 1 second.	Must	Timed test	Immediate feedback
NFR-PERF-003	Performance	A 5,000-row Binance CSV imports within 10 seconds without blocking the UI.	Should	Import test file	Active trader history
NFR-REL001	Reliability	No crash on empty data, network loss, malformed CSV or failed cloud calls.	Must	Failure test matrix	Brief §4.3
NFR-REL002	Reliability	A locally saved entry is never lost because sync failed.	Must	Interrupt sync	Trust in the data
NFR-DATA-001	Data integrity	Money stored as integer minor units (cents) plus currency code; no floatingpoint totals.	Must	Code review; totals exact	Statement must balance
NFR-DATA-002	Data integrity	After monthly close, opening + credits − debits = closing for every account.	Must	Statement check	Balance debit and credit
NFR-DATA-003	Data integrity	Stored exchange rates and LKR values never change retroactively.	Must	Rate-change test	AdSense accuracy
NFR-DATA-004	Data integrity	Backfilled and corrected records keep their real transaction date.	Must	Backfill previous month	Correct monthly reports
NFR-SEC001	Security	All financial screens and data require an authenticated Firebase user.	Must	Access without session	—

ID	Category	Requirement	Priority	Verification	Why (scenario)
NFR-SEC002	Security	Firestore Security Rules allow read/write only under users/{uid} where uid == request.auth.uid .	Must	Rules emulator tests	Private financial data
NFR-SEC003	Security	No hard-coded user IDs or data; all traffic over TLS.	Must	Code review	Brief marking rule
NFR-SEC004	Security	Optional PIN or biometric lock on app open.	Could	Enable lock	Shared or lost phone
NFR-PRIV-001	Privacy	No bank credentials, card numbers or Binance API keys are ever collected.	Must	Code review	Scope boundary
NFR-PRIV-002	Privacy	User can export all data as CSV and delete the account with all data.	Should	Export and delete	Data ownership
NFR-OFF001	Availability	Core entry and viewing work offline; sync is automatic on reconnect.	Must	Airplane-mode test	Play Store app had no sync
NFRCOMP-001	Compatibility	Installs and runs on API 26–35; notification permission requested at runtime on API 33+.	Must	Emulator API 26 and 34	Brief §5
NFRMAINT001	Maintainability	Strict MVVM; no business logic or data calls in Composables.	Must	Code review	Rubric criterion 2
NFRMAINT-002	Maintainability	ViewModels expose immutable StateFlow; mutable state stays private.	Must	Code review	Brief §5
NFRMAINT-003	Maintainability	Coroutines with viewModelScope; no blocking work on the main thread.	Must	StrictMode and code review	Brief §5
NFRMAINT-004	Maintainability	Hilt provides all ViewModels and repositories.	Must	Code review	Brief §5
NFRMAINT-005	Maintainability	FIFO P&L, FX, goal and reconciliation logic in unittested classes with ≥ 70% coverage.	Should	Coverage report	Correct numbers
ID	**Category**	**Requirement**	**Priority**	**Verification**	**Why (scenario)**
NFRSCALE001	Scalability	Queries bounded by date range and indexed; dashboard reads ≤ 50 documents per open.	Should	Firestore usage console	Cost and speed
NFR-LOC-001	Localisation	Amounts shown as LKR 123,456.00 by default; all strings externalised for future Sinhala and Tamil.	Should	Code review	Sri Lankan user
NFR-UX001	UX fit	No fixed salary or fixed monthly budget required to use the app.	Must	Use irregular dataset	Play Store app failure

Data Requirements

These are logical entities. The physical Firestore schema, indexes and security rules are separate deliverables but must be derived from this section. All records live under the owner's UID.

7.1 Logical entities

Entity	Key attributes	Notes
User	uid, email, baseCurrency, reminderTime, monthMode (calendar / salary cycle), createdAt	uid from Firebase Auth only
Account	accountId, name, type (Bank / Cash / Savings / Exchange), currency, openingBalance, isActive	Secondary bank is an Account, so its expenses are tracked

Entity	Key attributes	Notes
Transaction	txnId, type (Income / Expense / Transfer), amountMinor, currency, amountLkrMinor, fxRate, fxSource, date, accountId, toAccountId (transfers), categoryId, classification (Committed / Discretionary), incomeSource (Salary / Freelance / AdSense / Crypto), incomeBand (Dependable / Variable / Speculative), status (Expected / Confirmed), milestoneId, recurringId, isUntracked, note, syncState, createdAt, updatedAt	One collection for all money movements, so one date-range query feeds the dashboard and monthly close
Category	categoryId, name, icon, defaultClassification, useCount	useCount orders quick chips
SalaryProfile	payDay, baseAmount, bonusMin, bonusMax, accountId	One per user
Client	clientId, name, contact	—
Project	projectId, clientId, name, workType, totalValue, status, startDate, endDate	—
Milestone	milestoneId, projectId, name, amount, invoiceDate, dueDate, status (Planned / Invoiced / Paid / Overdue / Cancelled), paidAmount	Stored as a subcollection of Project
RecurringItem	recurringId, name, amount, currency, categoryId, accountId, cadence, nextDueDate, isVariable, isSubscription, lastReviewedAt, active	—
SavingsGoal	goalId, name, targetMinor, startAmountMinor, deadline, savingsAccountId, status, createdAt	Progress is derived, never stored as truth

Entity	Key attributes	Notes
CryptoTrade	tradeId, date, pair, side (Buy / Sell), price, quantity, fee, feeAsset, importBatchId	From Binance CSV

Entity	Key attributes	Notes
CryptoRealisedPnl	pnlId, tradeId, pnlUsdt, pnlLkrMinor, fxRate, date	Calculated by FIFO; recalculated after each import
CryptoSnapshot	snapshotId, date, valueUsdt, valueLkrMinor	Manual
MonthlyClose	closeId (YYYY-MM), perAccount [{accountId, calculatedMinor, actualMinor, differenceMinor}], status, closedAt	Differences are posted as Untracked transactions
ExchangeRate	date, base, quote, rate, source	Cached for offline suggestions

7.2 Multi-currency rules

- Every amount stores **original amount + currency**; LKR reporting value and the rate used are stored beside it.
- For AdSense, the user enters the **LKR actually credited**; effective rate = LKR ÷ USD.
- Stored LKR values are never recalculated retroactively.
- Amounts use integer minor units (cents) to avoid rounding errors.

7.3 Query and index requirements

Query need	Filter / order	Feeds
Transactions in a month	date range, order by date	Dashboard, statement, close
Confirmed income by source	type = Income, status = Confirmed, date range	Income chart
Expenses by classification	type = Expense, classification, date range	Committed vs discretionary
Expenses by category	type = Expense, categoryId, date range	Category chart
Query need	**Filter / order**	**Feeds**
Transactions by account	accountId, date range	Balance and reconciliation
Overdue milestones	status = Invoiced, dueDate < today	Follow-up reminders
Upcoming recurring items	active = true, order by nextDueDate	Dashboard, auto-entries
Crypto trades by pair	pair, order by date	FIFO calculation
Last entry date	order by createdAt desc, limit 1	Re-engagement scheduling

Composite indexes needed include (type, status, date), (type, classification, date), (type, categoryId, date) and (accountId, date). The schema document must justify each one.

System Architecture and Technical Constraints

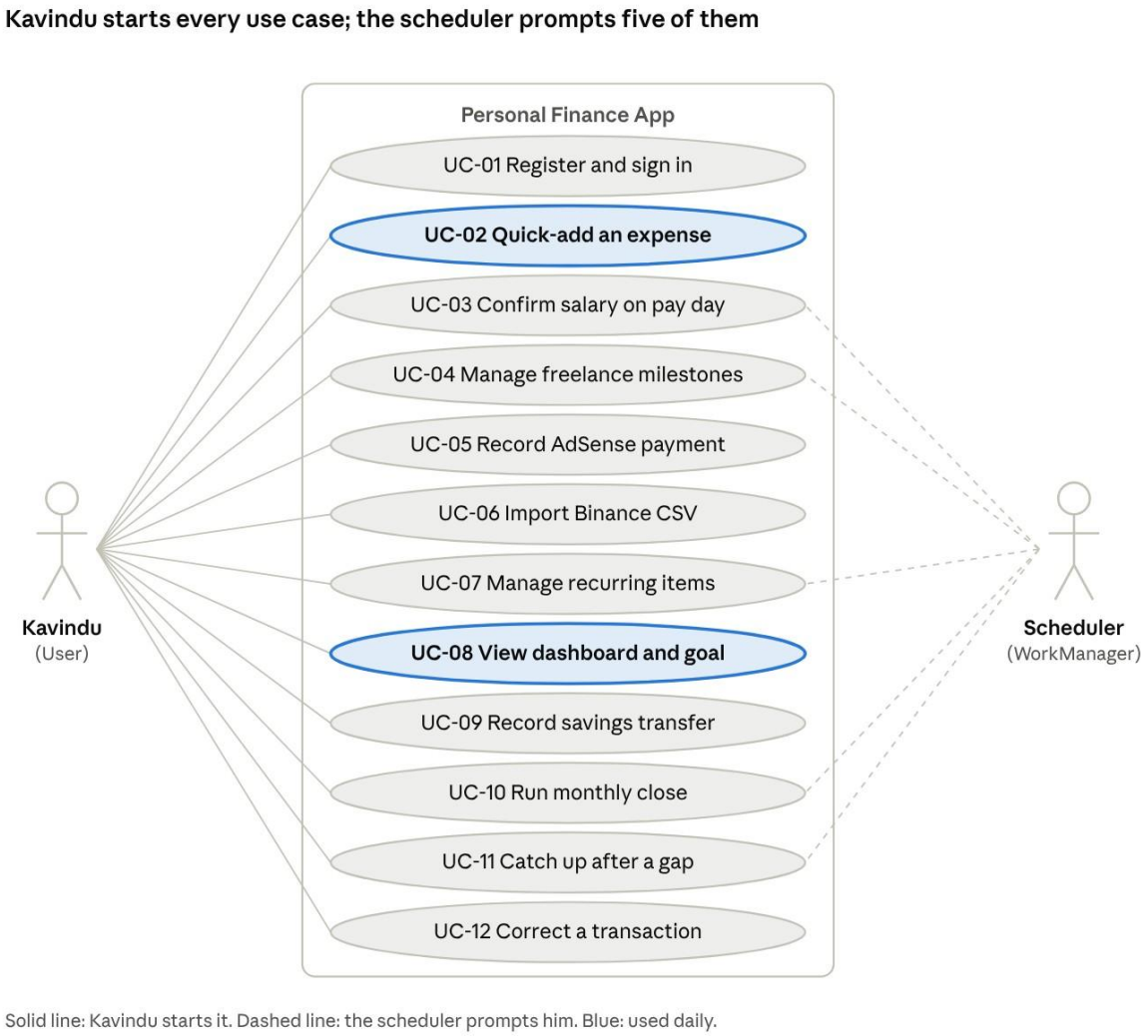
The app follows strict MVVM with a one-way dependency: **UI → ViewModel → Repository → data source**. The full MVVM diagram is a separate artefact (Section 13).

Layer / concern	Technology	Requirement
Language	Kotlin 100%, Gradle Kotlin DSL	No Java in the application layer
UI	Jetpack Compose, Material Design 3	No XML layouts; Composables render state and emit events only
Navigation	Navigation Compose, single activity	Typed arguments where practical
ViewModel	AndroidX ViewModel	Exposes immutable StateFlow<UiState> with Loading, Success, Empty and Error states
Async	Kotlin Coroutines, Flow, viewModelScope	No main-thread I/O
Domain logic	Use-case classes (e.g. CalculateRequiredSaving, ComputeFifoPnl, ReconcileMonth)	Pure Kotlin, unit-tested

Layer / concern	Technology	Requirement
Repository	One per area: Transaction, Account, Project, Recurring, Goal, Crypto, Close	Only layer that touches Room or Firestore
Local data	Room	Offline cache and pending-sync queue; UI reads from Room first
Cloud data	Cloud Firestore with offline persistence	Primary store; real-time listeners write into Room
Authentication	Firebase Auth (email/password; Google optional)	UID scopes every path
Background work	WorkManager	Pay-day prompts, recurring entries, overdue invoices, reengagement, month-end close, sync retry
File import	Storage Access Framework + CSV parser	No broad storage permission
DI	Hilt	No manual ViewModel creation in Composables
Security	Firestore Security Rules	users/{uid}/** readable and writable only by that uid
SDK	Min 26, target 34+	APK installs on Android 8.0+

Use Cases

9.1 Use case diagram



Use case diagram · 2 actors, 12 use cases

Kavindu is the only human actor. The scheduler (WorkManager) is a secondary actor that triggers pay-day, overdue-invoice, recurring, month-end and re-engagement prompts.

9.2 Use case summary

ID	Use case	Actors	Goal	Requirements
UC-01	Register and sign in	User	Secure personal account	FR-AUTH-001 to 007
UC02	Quick-add an expense	User	Record spending in 3 taps	FR-EXP-001 to 004
UC03	Confirm salary on pay day	User, Scheduler	Record the exact salary and bonus	FR-INC-004 to 006, FR-GOAL-005
UC04	Manage freelance milestones	User, Scheduler	Track invoices, chase overdue ones, link payments	FR-PROJ-001 to 006

ID	Use case	Actors	Goal	Requirements
UC05	Record AdSense payment	User	Store USD and LKR actually credited	FR-INC-007 to 010
UC06	Import Binance CSV	User	Know realised crypto P&L	FR-CRY-001 to 008
UC07	Manage recurring items	User, Scheduler	Automate rent, gym, subscriptions	FR-REC-001 to 004
UC08	View dashboard and goal	User	See the month and MacBook progress	FR-DASH-, *FR-GOAL-*
UC09	Record savings transfer	User	Pay himself first	FR-GOAL-005, FRACC-002
UC-10	Run monthly close	User, Scheduler	Balance the books	FR-CLOSE-001 to 005
UC-11	Catch up after a gap	User, Scheduler	Return without guilt	FR-ENG-002 to 004
UC-12	Correct a transaction	User	Fix category, account or date	FR-EXP-005

9.3 Detailed use cases

UC-02 Quick-add an expense

Element	Specification
Precondition	Signed in; at least one account exists
Main flow	1. Tap + on any main screen. 2. Keypad opens; enter amount. 3. Tap a category chip. 4. System saves with default account, today's date and category's classification. 5. Dashboard and safe-to-spend update; goal impact shown for discretionary spend.
Alternatives	Change account, date or add note before saving. Offline: saved to Room, marked pending.
Postcondition	Expense stored locally and synced to Firestore when online

UC-03 Confirm salary on pay day

Element	Specification
Precondition	Salary profile configured
Main flow	1. Scheduler creates Expected salary on the 25th and notifies. 2. User opens prompt; base pre-filled. 3. User enters exact credited amount; bonus calculated. 4. Status set to Confirmed. 5. System prompts "Move LKR 39,900 to savings now?" 6. User confirms after doing it in his bank app; transfer recorded.
Alternatives	Skip savings prompt (reminded next day). Not confirmed by the 28th: one reminder; stays Expected.
Postcondition	Confirmed salary and savings transfer recorded

UC-04 Manage freelance milestones

Element	Specification
Precondition	Signed in

Element	Specification
Element	**Specification**
Main flow	1. Create client and project. 2. Add milestones with amounts and due dates. 3. Mark milestone Invoiced. 4. Scheduler marks it Overdue after due date and reminds weekly. 5. On payment, user records income and selects the milestone. 6. Milestone set to Paid; outstanding total updates.
Alternatives	Partial payment leaves remainder outstanding. Payment with no milestone saved as unlinked freelance income.
Postcondition	Every freelance payment traceable to a project

UC-06 Import Binance CSV

Element	Specification
Precondition	User exported trade history from Binance
Main flow	1. Open Crypto → Import. 2. Pick CSV file. 3. System validates columns and shows preview (read / rejected / duplicate). 4. User confirms. 5. System stores trades and recalculates FIFO realised P&L. 6. YTD and monthly P&L shown.
Alternatives	Wrong format: rejected with missing columns listed. Sell with unknown cost: flagged.
Postcondition	Crypto P&L known; kept apart from dependable income

UC-10 Run monthly close

Element	Specification
Precondition	Month has ended
Element	**Specification**
Main flow	1. Scheduler prompts on the 1st. 2. User enters real balance for each account. 3. System shows calculated vs actual difference. 4. User adds missing entries or records Unaccounted. 5. System generates balanced monthly statement.
Alternatives	Skip: reminder after 3 days; month remains open.
Postcondition	Every account balances; statement saved

UC-11 Catch up after a gap

Element	Specification
Precondition	No entries for 3+ days
Main flow	1. Scheduler sends re-engagement notification. 2. User opens app; Catch-up mode starts. 3. Confirm pending recurring items and salary. 4. Add remembered large expenses (optional). 5. Enter current balances. 6. Remaining difference posted as Untracked per account. 7. Dashboard is current again.
Alternatives	User dismisses catch-up; can open it later from the dashboard.
Postcondition	Balances correct; gap visible but not blocking

Error Handling and Edge Cases

Case	Required behaviour
Negative, zero or malformed amount	Reject; highlight the field
Missing category or account	Block save; name the missing field
No network during save	Save to Room; mark pending; sync later
Sync fails repeatedly	Keep record; show failed state with retry

Case	Required behaviour
Exchange-rate API unavailable	Use last cached rate as suggestion; allow manual rate
AdSense LKR credited differs from market rate	Store effective rate from actual credit
Salary not confirmed by the 28th	Keep as Expected; send one reminder; excluded from confirmed totals
Freelance payment with no matching milestone	Offer to create milestone, or save as unlinked freelance income
Partial milestone payment	Record payment; remaining amount stays outstanding
Binance CSV with wrong or missing columns	Reject file; list missing columns
Binance CSV re-imported	Skip duplicates; report count
Sell trade with no prior buy in data	Flag "cost basis unknown"; exclude from P&L until user enters cost
Crypto loss	Show as negative P&L; never reduce dependable income
Transfer to own account	Never counted as expense or income
Goal deadline passed but not reached	Offer to extend deadline or raise monthly saving
Contributions exceed target	Mark goal Completed; show 100%
Long gap (14+ days)	Catch-up mode; post Untracked difference per account
Monthly close difference	Add entries or record Unaccounted; never hidden
Zero committed expenses	Ratio shows "no committed costs yet" instead of dividing by zero
Empty dataset on any screen	Empty state with next action
Recurring template cancelled	No new entries; history kept

Case	Required behaviour
Rapid double tap on save	One record only

Requirements Traceability Matrix

Every problem from Section 2.6 maps to requirements, a design artefact and a test.

Problem	Requirements	Design artefact	Verification
P1 Financial invisibility	FR-ACC-004, FR-CLOSE-004, FR-DASH-001/008, NFRUSE-004	Dashboard wireframe	Dashboard answers earned/spent/saved
P2 Income estimated, not confirmed	FR-INC-001 to 011, FR-DASH005	Transaction schema (status, fx fields)	Expected vs confirmed totals; USD effective-rate test
P3 Freelance reconciliation	FR-PROJ-001 to 006, FR-DASH007	Project/milestone schema; project screen	Overdue reminder; payment linked to milestone
P4 Crypto result unknown	FR-CRY-001 to 008	CryptoTrade/PnL schema; import screen	FIFO P&L on sample CSV; YTD gain and loss
P5 Expense fragmentation	FR-ACC-001 to 003, FR-EXP-002/005, FRREC-001 to 004	Account model	Cash, card and secondary-account totals
P6 No committed/discretionary view	FR-EXP-003, FR-DASH-003/004/006, FR-GOAL-006	Insights wireframe	Known-data ratio check

Problem	Requirements	Design artefact	Verification
P7 Tracking friction	FR-EXP-001/004/008, FR-REC-002, FR-ENG-001, NFR-USE-001	Quick-add wireframe	Tap count and timing
P8 No recovery after gaps	FR-EXP-007, FR-ENG-002 to 004, NFR-USE003/005	Catch-up flow wireframe	14-day gap catch-up under 2 minutes
P9 Goal disconnected	FR-GOAL-001 to 008, FR-CRY008	Goal wireframe	478,800 remaining; 39,900/month; projection
P10 Books never verified	FR-CLOSE-001 to 005, NFR-DATA-002	MonthlyClose schema	Statement balances for every account
Firebase Auth and Rules	FR-AUTH-001 to 005, NFR-SEC001 to 003	firestore.rules	Emulator cross-user test
Offline and sync	FR-OFF-001 to 005, NFR-OFF001, NFR-REL002	Room entities, sync worker	Airplane-mode test

Problem	Requirements	Design artefact	Verification
Architecture constraints	C-01 to C-12, NFR-MAINT-001 to 005	MVVM diagram, repository	Code review
Error and empty states	FR-VAL-001 to 005, NFR-REL-001	UiState design	Failure test matrix

Verification and Acceptance

12.1 Acceptance evidence

Area	Minimum evidence for the demo
Income	All four sources; salary confirmed from pay-day prompt; USD AdSense with effective rate; expected vs confirmed totals
Freelance	Project with one Paid, one Invoiced and one Overdue milestone; reminder fired
Crypto	Sample Binance CSV imported; duplicate re-import skipped; YTD realised P&L with a gain and a loss
Expenses	Card, cash and secondary-account expenses; 3tap entry; correction; backfill
Recurring	Rent and subscription templates; auto-entry confirmed; subscription review
Goal	MacBook goal: 478,800 remaining, about 39,900/month, projected date, savings transfer prompt
Monthly close	Real balances entered; difference recorded as Unaccounted; balanced statement
Engagement	Re-engagement notification; catch-up after simulated gap
Security	Two accounts cannot read each other's data
Offline	Entry in airplane mode; sync on reconnect
Architecture	MVVM, StateFlow, Coroutines, Hilt, Navigation Compose, single activity
Compatibility	APK runs on API 26 and API 34

12.2 Formula definitions

Metric	Definition
Confirmed income	Σ Income where status = Confirmed and date in period (LKR reporting values)
Expected income	Σ Income where status = Expected and date in period; shown separately
Expenses	Σ Expense where date in period; transfers excluded
Committed / discretionary	Σ Expense by classification in period
Discretionary ratio	Discretionary ÷ (committed + discretionary)
Net cash flow	Confirmed income − expenses
Safe to spend	Confirmed income − committed − planned saving − discretionary spent
Goal remaining	max(target − saved, 0)
Required monthly saving	Goal remaining ÷ whole months left (minimum 1)
Projected completion	Today + goal remaining ÷ average saving of last 3 months
Goal delay (days)	Expense ÷ (required monthly saving ÷ 30)
Realised crypto P&L	Σ (sell proceeds − FIFO cost of units sold − fees)
Account balance	Opening + credits − debits (incl. transfers)
Unaccounted	Actual balance − calculated balance at monthly close

For the scenario, required saving = (490,000 − 11,200) ÷ 12 = **LKR 39,900 per month**.

Related Assignment Artefacts

These deliverables are separate from this SRS but must stay consistent with it.

Artefact	Must show	Traces to
Firestore schema and indexes	Collections under users/{uid} , multi-currency fields, composite indexes with justification	Section 7
firestore.rules	Per-user isolation, field validation	FR-AUTH-004, NFR-SEC002
MVVM architecture diagram	Screens → ViewModels → Repositories → Room/Firestore	Section 8
Annotated wireframes (minimum 3)	Dashboard, Quick-add expense, Goal; plus Catch-up and Monthly close recommended	FR-EXP-001, FR-DASH-001, FR-GOAL-*, FR-ENG-003
README.md	Overview, Firebase setup, build steps, architecture summary	Brief §6.1
APK	Connects to live Firebase; runs on Android 8.0+	NFR-COMP-001
Live demo	Core flows plus questions on ViewModels, queries, requirement derivation	Section 12

Conclusion

This SRS specifies a focused app for one problem: Kavindu earns well but cannot see his money. It separates confirmed from expected income, realised crypto P&L from portfolio change, and transfers from spending. It makes every routine action take seconds and lets the monthly close balance the books even after missed days. Each requirement traces to a specific failure in the scenario, so the result is something Kavindu would still open daily six months from now.

5.7 Recurring transactions (REC)

ID	Requirement	Rule	Priority	Verification	Source
FRREC-001	The system shall let the user create recurring templates (rent, utilities, gym, subscriptions) with amount, category, account, cadence and next due date.	User-created only; no bank import	Must	Create rent template	P5
FRREC-002	On the due date the system shall create the entry for one-tap confirmation or amount adjustment.	Variable items (utilities) open the amount field	Must	Due date passes; entry pending confirmation	P7
FRREC-003	The system shall list active subscriptions with monthly and yearly cost and prompt a "still using this?" review every 3 months.	App never cancels services	Should	Review prompt appears	P5
FRREC-004	Editing or cancelling a template shall not change past entries created from it.	Cancelled templates create no new entries	Must	Cancel; history intact	—

5.8 Savings goal (GOAL)

ID	Requirement	Rule	Priority	Verification	Source
FRGOAL-001	The system shall let the user create a goal with name, target amount, deadline, starting amount and linked Savings account.	Target > 0; deadline in future; start ≥ 0	Must	Create MacBook goal: 490,000, 12 months, 11,200	P9
FRGOAL-002	The system shall calculate amount remaining and required monthly saving = remaining ÷ months left.	Recalculates on any change; never hardcoded	Must	Shows 478,800 and about 39,900/month	P9
ID	**Requirement**	**Rule**	**Priority**	**Verification**	**Source**
FRGOAL-003	The system shall show progress as amount and percentage.	Capped at 100% for display	Must	Below, at and above target	P9
FRGOAL-004	The system shall project a completion date from the average actual saving of the last 3 months.	Shows "not enough data" before month 1	Must	Known contributions give expected date	P9
FRGOAL-005	When salary is confirmed, the system shall prompt "Move LKR X to savings now?" and record the transfer when the user confirms he has done it.	X = required monthly saving; editable	Must	Confirm salary; prompt; transfer recorded	P9

ID	Requirement	Rule	Priority	Verification	Source
FRGOAL-006	After a discretionary expense is saved, the system shall show its goal impact in days of delay.	Informational only; no advice	Should	Known expense shows correct days	P6, P9
FRGOAL-007	The system shall show a chart of actual vs required monthly saving.	—	Must	Chart matches records	P9
FRGOAL-008	The system shall notify at 25%, 50%, 75% and 100% of the target.	Once per milestone	Should	Cross 25%; notification	P9

5.9 Monthly close and reconciliation (CLOSE)

ID	Requirement	Rule	Priority	Verification	Source
FR-CLOSE001	At month end the system shall prompt the user to enter each account's real balance.	Month = calendar month by default; salary cycle (25th–24th) optional	Must	Month-end prompt appears	P10
FR-CLOSE-002	The system shall show the difference between real and calculated balance for each account.	—	Must	Known gap shown	P10
ID	**Requirement**	**Rule**	**Priority**	**Verification**	**Source**
FR-CLOSE-003	The user shall resolve each difference by adding missing entries or recording it as Unaccounted.	Unaccounted is visible, never hidden	Must	Record Unaccounted; balances match	P1, P10
FR-CLOSE-004	The system shall produce a monthly statement: income by source, expenses by category, transfers, savings, Unaccounted and closing balances.	Opening + credits − debits = closing for every account	Must	Statement balances to zero	P1, P2
FR-CLOSE-005	The system shall show the Unaccounted amount over months as a measure of tracking quality.	—	Should	Trend chart shown	P10

5.10 Dashboard and financial visibility (DASH)

ID	Requirement	Rule	Priority	Verification	Source
FRDASH-001	The dashboard shall show, for the current period, confirmed income, expected income, expenses, net cash flow, safe-to-spend and goal progress.	Labels state clearly what each figure is	Must	Populated and empty dashboard	P1
FRDASH-002	The dashboard shall update through Firestore real-time listeners.	Scoped to current user	Must	Edit on device B; device A updates	—
FRDASH-003	The system shall show spending by category for a selected period.	—	Must	Chart matches records	P6
FRDASH-004	The system shall show committed vs discretionary totals and ratio.	Zero committed handled without error	Must	Known data gives known ratio	P6
FRDASH-005	The system shall show income by source for the current month and last 6 months.	—	Must	Chart matches records	P2

ID	Requirement	Rule	Priority	Verification	Source
FRDASH-006	Safe-to-spend = confirmed income − committed expenses − planned saving − discretionary spent.	Can be negative; shown in warning colour	Should	Known values give expected figure	P6, P9
FRDASH-007	The system shall show outstanding freelance amount and next recurring items due.	—	Must	Values visible on dashboard	P3, P5
FRDASH-008	The system shall show a weekly summary from day 7, even with partial data.	—	Should	Summary after 7 days	P1

5.11 Engagement and re-entry (ENG)

ID	Requirement	Rule	Priority	Verification	Source
FRENG-001	The system shall send a daily reminder at a user-chosen time if no entry was made that day.	Off by default after 3 ignored days	Must	No entry; reminder fires	P7
FRENG-002	After 3 and 7 days without entries the system shall send a re-engagement notification; after 14 days a final "catch up in 2 minutes" message, then stop.	Friendly wording; no guilt	Must	Simulate gaps	P8
FRENG-003	On return after 3+ days, the system shall offer Catch-up mode: confirm pending recurring items and salary, add remembered large expenses, enter current balances, and record the remainder as Untracked per account.	Completes in under 2 minutes	Must	14-day gap; balances correct after catch-up	P8
FRENG-004	The system shall show a tracking streak and never mark missed days as failures.	—	Should	Gap shown neutrally	P8
ID	**Requirement**	**Rule**	**Priority**	**Verification**	**Source**
FRENG-005	The user shall be able to switch each notification type on or off.	—	Must	Toggle each type	—

5.12 Offline operation and synchronisation (OFF)

ID	Requirement	Rule	Priority	Verification	Source
FR-OFF001	The system shall allow adding, editing and viewing records with no network.	UI never blocks on Firestore	Must	Airplane mode; add expense	P7
FR-OFF002	Records shall be stored in Room and survive app restarts while offline.	Tied to the current user	Must	Restart offline; data present	—
FR-OFF003	Pending changes shall sync to Firestore when connectivity returns.	Failed sync retried; never discarded	Must	Reconnect; data in Firestore	—
FR-OFF004	The UI shall show pending, synced and failed sync states.	—	Should	Simulate failure; indicator shown	—

ID	Requirement	Rule	Priority	Verification	Source
FR-OFF005	Multi-device conflicts are out of scope; the app shall flag rather than silently overwrite an unsynced local record.	Last-write-wins only for synced records	Should	Document and test boundary	—

5.13 Data validation and error handling (VAL)

ID	Requirement	Rule	Priority	Verification	Source
FR-VAL001	The system shall block saving records with missing required fields and name the missing field.	—	Must	Submit incomplete form	—
FR-VAL002	The system shall reject zero, negative or malformed amounts and rates.	—	Must	Boundary tests	—
FR-VAL003	The system shall prevent duplicate entries from repeated taps or slow network.	One action creates one record	Should	Rapid double tap	—

ID	Requirement	Rule	Priority	Verification	Source
FR-VAL004	Every major screen shall show an explicit empty state with a next action.	No blank screens	Must	New user walkthrough	P1
FR-VAL005	Cloud failures shall show a recoverable error while local data stays visible.	No crash, no blank screen	Must	Kill network during save	—